

**UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN**



CURSO: Base de Datos II

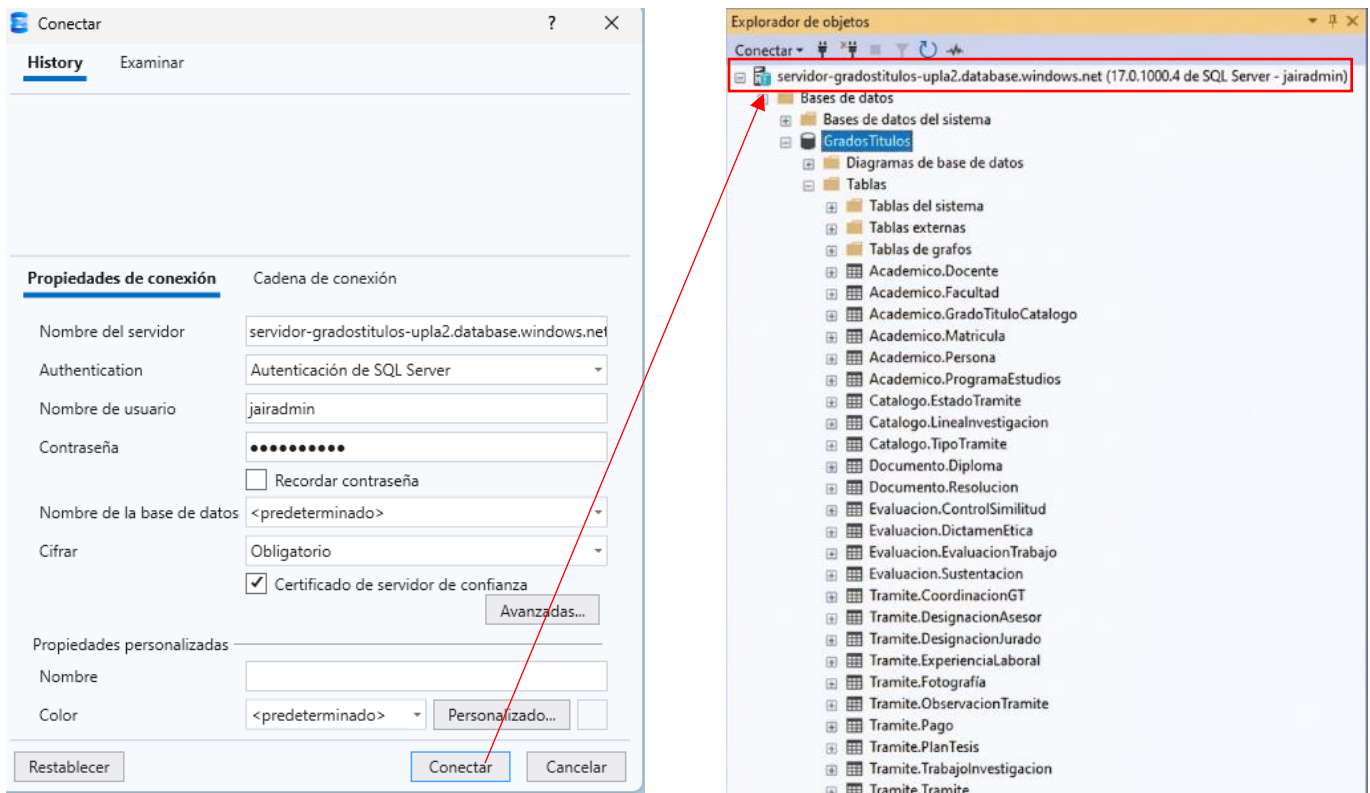
DOCENTE: Ing. Fernandez Bejarano Raul Enrique

ESTUDIANTE: Nuñez Michel Jair

CICLO: V

SECCION: B1

Conectar Azure SQL en SQL Server Management Studio 22 y desarrollo de todos los enunciados y ejercicios:



TEMA 1: Estrategias de Backup : completo, Diferencial y Transaccional

Proyecto 1: Respaldo Completo Institucional de la Base de Datos GradosTitulos Enunciado

La Oficina de Tecnologías de Información de la universidad requiere implementar una política de respaldo completo diario para la base de datos **GradosTitulos**, la cual almacena información crítica de estudiantes, trámites, evaluaciones, resoluciones y diplomas.

El estudiante deberá crear un procedimiento que genere un respaldo completo de la base de datos y registre la fecha de ejecución.

```
USE GradosTitulos;
GO

-- Crear la tabla para registrar los respaldos realizados
CREATE TABLE dbo.HistorialBackups
(
    IdBackup INT IDENTITY(1,1) PRIMARY KEY,
    TipoBackup VARCHAR(30),
    FechaBackup DATETIME,
    RutaArchivo VARCHAR(255)
);
GO

-- Crear el procedimiento almacenado
CREATE PROCEDURE SP_BackupCompletoGradosTitulos
AS
BEGIN
    BACKUP DATABASE GradosTitulos
    TO DISK = 'D:\Backups\GradosTitulos_Full.bak'
    WITH INIT,
    COMPRESSION,
    STATS = 10;

    INSERT INTO dbo.HistorialBackups
    (
        TipoBackup,
        FechaBackup,
        RutaArchivo
    )
    VALUES
    (
        'FULL',
        GETDATE(),
        'D:\Backups\GradosTitulos_Full.bak'
    );
END;
GO

-- Ejecutar el procedimiento
EXEC SP_BackupCompletoGradosTitulos;
GO

-- Verificar el historial de respaldos
SELECT *
FROM dbo.HistorialBackups;
GO
```

Justificación técnica de la solución aplicada

Se implementó un procedimiento almacenado que automatiza la generación del respaldo completo de la base de datos y registra cada ejecución en una tabla de historial para facilitar el seguimiento de los respaldos realizados.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó un procedimiento almacenado para centralizar el proceso de respaldo, se habilitó la compresión para optimizar el espacio de almacenamiento y se registró la fecha de cada ejecución para mantener un control de las copias de seguridad.

Respaldo y Recuperación de Bases de Datos

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos Enunciado

La universidad necesita reducir los tiempos de respaldo debido al crecimiento de las tablas de los esquemas **Tramite** y **Documento**.

Diseñe una estrategia que combine respaldos completos semanales y diferenciales diarios.

Script de la solución en T-SQL

```
USE master;
GO

-- Respaldo completo semanal
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Full_Semanal.bak'
WITH INIT,
    COMPRESSION,
    STATS = 10;
GO

-- Respaldo diferencial diario
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    STATS = 10;
GO

USE GradosTitulos;
GO

-- Registrar los respaldos realizados
INSERT INTO dbo.HistorialBackups
(
    TipoBackup,
    FechaBackup,
    RutaArchivo
)
VALUES
('FULL', GETDATE(), 'D:\Backups\GradosTitulos_Full_Semanal.bak'),
('DIFFERENTIAL', GETDATE(), 'D:\Backups\GradosTitulos_Diferencial.bak');
GO

-- Verificar el historial de respaldos
SELECT *
FROM dbo.HistorialBackups;
GO
```

Justificación técnica de la solución aplicada

Se implementó una estrategia de respaldo compuesta por un respaldo completo semanal y respaldos diferenciales diarios, reduciendo el tiempo necesario para generar las copias de seguridad y optimizando el uso del almacenamiento.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó la compresión de respaldos para ahorrar espacio, se registró cada copia en una tabla de historial y se separaron los respaldos completos y diferenciales para facilitar la recuperación de la base de datos.

Proyecto 3: Estrategia Integral Full + Differential + Log Backup

Enunciado

La base de datos **GradosTitulos** opera durante las 24 horas y registra continuamente solicitudes de grados y títulos.

Diseñe una estrategia que contemple:

- Respaldo completo semanal.
- Respaldo diferencial diario.
- Respaldo del log de transacciones cada 30 minutos.

Script de la solución en T-SQL

```
USE master;
GO

-- Respaldo completo semanal
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Full.bak'
WITH INIT,
    COMPRESSION,
    STATS = 10;
GO

-- Respaldo diferencial diario
BACKUP DATABASE GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    STATS = 10;
GO

-- Respaldo del log de transacciones (cada 30 minutos)
BACKUP LOG GradosTitulos
TO DISK = 'D:\Backups\GradosTitulos_Log.trn'
WITH INIT,
    COMPRESSION,
    STATS = 10;
GO

USE GradosTitulos;
GO

-- Registrar los respaldos realizados
INSERT INTO dbo.HistorialBackups
(
    TipoBackup,
    FechaBackup,
    RutaArchivo
)
VALUES
('FULL', GETDATE(), 'D:\Backups\GradosTitulos_Full.bak'),
('DIFFERENTIAL', GETDATE(), 'D:\Backups\GradosTitulos_Diferencial.bak'),
('LOG', GETDATE(), 'D:\Backups\GradosTitulos_Log.trn');
GO

-- Verificar el historial de respaldos
SELECT *
FROM dbo.HistorialBackups;
GO
```

Justificación técnica de la solución aplicada

Se implementó una estrategia de respaldo compuesta por copias completas, diferenciales y del log de transacciones, permitiendo reducir la pérdida de información y agilizar la recuperación de la base de datos ante cualquier incidente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se combinaron distintos tipos de respaldo para optimizar el tiempo de recuperación, se utilizó compresión para disminuir el espacio ocupado por las copias de seguridad y se registró cada respaldo en una tabla de historial para facilitar su administración.

Proyecto 4: Diseño de Estrategia de Respaldo para Alta Disponibilidad

Enunciado

Se requiere diseñar una estrategia de recuperación ante desastres para la plataforma institucional de grados y títulos.

El estudiante deberá definir respaldos completos, diferenciales y transaccionales para garantizar una recuperación máxima de **15 minutos**.

Script de la solución en T-SQL

```
USE master;
GO

-- Respaldo completo
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Respaldo diferencial
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Respaldo del log de transacciones
BACKUP LOG GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

USE GradosTitulos;
GO

-- Registrar los respaldos realizados
INSERT INTO dbo.HistorialBackups
(
    TipoBackup,
    FechaBackup,
    RutaArchivo
)
VALUES
('FULL', GETDATE(), 'D:\Backups\GradosTitulos_Full.bak'),
('DIFFERENTIAL', GETDATE(), 'D:\Backups\GradosTitulos_Diferencial.bak'),
('LOG', GETDATE(), 'D:\Backups\GradosTitulos_Log.trn');
GO

-- Verificar el historial de respaldos
SELECT *
FROM dbo.HistorialBackups;
GO
```

Justificación técnica de la solución aplicada

Se implementó una estrategia de respaldo que combina copias completas, diferenciales y del log de transacciones, permitiendo recuperar la base de datos con una pérdida mínima de información y cumplir con un objetivo de recuperación de hasta 15 minutos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se incorporó la compresión y la verificación mediante **CHECKSUM** para mejorar la confiabilidad de los respaldos. Además, se registró cada copia en el historial para facilitar el control y la administración del plan de recuperación.

Proyecto 5: Implementación de una Estrategia Corporativa de Respaldo para GradosTítulos

Enunciado

La Universidad ha identificado que la base de datos **GradosTítulos** es crítica para la gestión de expedientes de grados y títulos. Debido al incremento de trámites registrados en los esquemas

Academico, Tramite, Evaluacion y Documento, se requiere implementar una estrategia avanzada de respaldo que garantice una pérdida máxima de información de **15 minutos**.

Diseñe una solución que incluya:

- Respaldo completo cada domingo.
- Respaldo diferencial diario.
- Respaldo del log de transacciones cada 15 minutos.
- Compresión de respaldos.
- Verificación automática de integridad.

Script de la solución en T-SQL

```
USE master;
GO

-- Respaldo completo (Domingos)
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Full_Domingo.bak'
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Respaldo diferencial (Diario)
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Respaldo del Log de Transacciones (Cada 15 minutos)
BACKUP LOG GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Verificar la integridad del respaldo completo
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos_Full_Domingo.bak';
GO

USE GradosTitulos;
GO

-- Registrar los respaldos realizados
INSERT INTO dbo.HistorialBackups
(
    TipoBackup,
    FechaBackup,
    RutaArchivo
)
VALUES
('FULL', GETDATE(), 'D:\Backups\GradosTitulos_Full_Domingo.bak'),
('DIFFERENTIAL', GETDATE(), 'D:\Backups\GradosTitulos_Diferencial.bak'),
('LOG', GETDATE(), 'D:\Backups\GradosTitulos_Log.trn');
GO

-- Verificar el historial de respaldos
SELECT *
FROM dbo.HistorialBackups;
GO
```

Justificación técnica de la solución aplicada

Se implementó una estrategia corporativa basada en respaldos completos, diferenciales y del log de transacciones, incorporando compresión y verificación de integridad para reducir la pérdida de información y garantizar la confiabilidad de las copias de seguridad.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las opciones **COMPRESSION** y **CHECKSUM** para optimizar el almacenamiento y detectar posibles errores durante el respaldo. Además, se verificó la integridad mediante **RESTORE VERIFYONLY** y se registró cada respaldo en el historial para facilitar su administración y seguimiento.

Proyecto 6: Diseño de Estrategia de Respaldo para Recuperación ante Desastres

Enunciado

La Oficina de Tecnologías de Información requiere implementar una estrategia de **Disaster Recovery** para la base de datos **GradosTitulos**.

Diseñe un esquema de respaldo que permita reconstruir completamente la base de datos en un servidor alternativo ante la pérdida total del servidor principal.

Script de la solución en T-SQL

```
USE master;
GO

-- Respaldo completo de la base de datos
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_DR_Full.bak'
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Respaldo diferencial
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_DR_Diferencial.bak'
WITH DIFFERENTIAL,
    INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Respaldo del log de transacciones
BACKUP LOG GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_DR_Log.trn'
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Verificar la integridad del respaldo
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos_DR_Full.bak';
GO

-- Restauración en un servidor alternativo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_DR_Full.bak'
WITH NORECOVERY;
GO

RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_DR_Diferencial.bak'
WITH NORECOVERY;
GO

RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_DR_Log.trn'
WITH RECOVERY;
GO

USE GradosTitulos;
GO

-- Registrar el respaldo realizado
INSERT INTO dbo.HistorialBackups
(
    TipoBackup,
    FechaBackup,
    RutaArchivo
)
VALUES
('FULL', GETDATE(), 'D:\Backups\GradosTitulos_DR_Full.bak'),
('DIFFERENTIAL', GETDATE(), 'D:\Backups\GradosTitulos_DR_Diferencial.bak'),
('LOG', GETDATE(), 'D:\Backups\GradosTitulos_DR_Log.trn');
GO

-- Verificar el historial de respaldos
SELECT *
FROM dbo.HistorialBackups;
GO
```

Justificación técnica de la solución aplicada

Se diseñó una estrategia de **Disaster Recovery** utilizando respaldos completos, diferenciales y del log de transacciones, permitiendo reconstruir la base de datos en un servidor alternativo con la menor pérdida posible de información.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron respaldos comprimidos, verificación de integridad mediante **RESTORE VERIFYONLY** y una secuencia correcta de restauración (**FULL** → **DIFFERENTIAL** → **LOG**) para garantizar una recuperación segura y confiable de la base de datos.

TEMA 2. Restauración de Bases de Datos en Distintos Escenarios

Proyecto 1: Restauración de registros eliminados accidentalmente

Enunciado

Un administrador eliminó accidentalmente registros relacionados con expedientes de titulación. Restaure la base de datos utilizando el respaldo más reciente.

Script de la solución en T-SQL

```
USE master;
GO

-- Restaurar la base de datos desde el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH REPLACE,
RECOVERY,
STATS = 10;
GO

-- Verificar que la base de datos se restauró correctamente
USE GradosTitulos;
GO

SELECT
    name AS NombreBaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO
```

Justificación técnica de la solución aplicada

Se restauró la base de datos utilizando el respaldo completo más reciente, recuperando los registros eliminados accidentalmente y devolviendo la base de datos a un estado consistente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó la opción **WITH REPLACE** para sobrescribir la base de datos existente y **WITH RECOVERY** para dejarla disponible inmediatamente después de la restauración. Finalmente, se verificó el estado de la base para confirmar que el proceso concluyó correctamente.

Proyecto 2: Restauración en un Servidor Alternativo

Enunciado

La universidad requiere habilitar un servidor de contingencia para pruebas y recuperación. Restaure la base de datos **GradosTitulos** en una nueva instancia SQL Server.

Script de la solución en T-SQL

```
USE master;
GO

-- Restaurar la base de datos en el servidor alternativo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
MOVE 'GradosTitulos_Data'
TO 'C:\SQLData\GradosTitulos_Data.mdf',

MOVE 'GradosTitulos_Log'
TO 'C:\SQLLog\GradosTitulos_Log.ldf',

RECOVERY,
STATS = 10;
GO

-- Verificar la restauración
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Consultar los archivos físicos restaurados
USE GradosTitulos;
GO

SELECT
    name AS NombreLogico,
    physical_name AS RutaFisica,
    type_desc AS TipoArchivo
FROM sys.database_files;
GO
```


Justificación técnica de la solución aplicada

Se restauró la base de datos en un servidor alternativo utilizando la opción **MOVE** para definir nuevas rutas físicas de los archivos de datos y del log, permitiendo la recuperación del sistema en un entorno diferente al servidor original.

Explicación de las buenas prácticas utilizadas en el proyecto

Se especificaron nuevas ubicaciones para los archivos mediante **MOVE**, evitando conflictos con archivos existentes. Además, se verificó el estado de la base de datos y la ubicación de los archivos restaurados para confirmar que la recuperación fue exitosa.

Proyecto 3: Recuperación después de Corrupción del Archivo MDF

Enunciado

El archivo principal de datos presenta corrupción física.

Implemente el procedimiento de restauración completo utilizando los respaldos disponibles.

Script de la solución en T-SQL

```
USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH NORECOVERY,
    REPLACE,
    STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH NORECOVERY,
    STATS = 10;
GO

-- Restaurar el respaldo del log de transacciones
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH RECOVERY,
    STATS = 10;
GO

-- Verificar el estado de la base de datos
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO
```

Justificación técnica de la solución aplicada

Se restauró la base de datos siguiendo la secuencia **respaldo completo, respaldo diferencial y respaldo del log de transacciones**, permitiendo recuperar la información después de la corrupción del archivo **MDF** y garantizar la consistencia de los datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó **NORECOVERY** durante las restauraciones intermedias y **RECOVERY** al finalizar el proceso para dejar la base operativa. Finalmente, se ejecutó **DBCC CHECKDB** para comprobar la integridad física y lógica de la base de datos restaurada.

Proyecto 4: Recuperación ante Desastre Total del Servidor

Enunciado

Una falla de almacenamiento destruyó completamente el servidor donde se encontraba la base **GradosTitulos**.

Diseñe el proceso de recuperación completa.

Script de la solución en T-SQL

```
USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH NORECOVERY,
     MOVE 'GradosTitulos_Data'
     TO 'C:\SQLData\GradosTitulos_Data.mdf',

     MOVE 'GradosTitulos_Log'
     TO 'C:\SQLLog\GradosTitulos_Log.ldf',

     STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH NORECOVERY,
     STATS = 10;
GO

-- Restaurar el respaldo del log de transacciones
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH RECOVERY,
     STATS = 10;
GO

-- Verificar el estado de la base restaurada
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base de datos
DBCC CHECKDB ('GradosTitulos');
GO
```

Justificación técnica de la solución aplicada

Se implementó un proceso de recuperación ante desastres restaurando los respaldos completo, diferencial y del log de transacciones en un nuevo servidor, permitiendo reconstruir completamente la base de datos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las opciones **MOVE** para ubicar los archivos en nuevas rutas, **NORECOVERY** durante las restauraciones intermedias y **RECOVERY** al finalizar el proceso. Finalmente, se verificó la integridad de la base mediante **DBCC CHECKDB** para asegurar una recuperación correcta.

Proyecto 5: Recuperación Completa después de Corrupción del Archivo MDF Enunciado

Se detectó corrupción física en el archivo principal de datos de la base **GradosTitulos**, impidiendo el acceso a las tablas relacionadas con expedientes de titulación. Recupere completamente la base utilizando la secuencia de respaldos disponibles.

Script de la solución en T-SQL

```
USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH REPLACE,
     NORECOVERY,
     STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH NORECOVERY,
     STATS = 10;
GO

-- Restaurar el primer respaldo del Log
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log1.trn'
WITH NORECOVERY,
     STATS = 10;
GO

-- Restaurar el último respaldo del Log
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log2.trn'
WITH RECOVERY,
     STATS = 10;
GO

-- Verificar el estado de la base restaurada
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base de datos
DBCC CHECKDB ('GradosTitulos');
GO
```

Justificación técnica de la solución aplicada

Se recuperó completamente la base de datos aplicando la secuencia correcta de restauración: respaldo completo, respaldo diferencial y respaldos del log de transacciones. Esto permitió restaurar la información hasta el último punto disponible antes de la corrupción del archivo MDF.

Explicación de las buenas prácticas utilizadas en el proyecto

Se empleó NORECOVERY durante las restauraciones intermedias y RECOVERY únicamente en el último respaldo del log para completar el proceso. Finalmente, se verificó la integridad de la base con DBCC CHECKDB, asegurando que la recuperación fuera correcta.

Proyecto 6: Restauración de la Base de Datos en un Servidor de Contingencia

Enunciado

La Universidad requiere habilitar un centro alternativo de recuperación para continuar los procesos de grados y títulos.

Restaure la base de datos **GradosTítulos** en una instancia secundaria.

Script de la solución en T-SQL

```
USE master;
GO

-- Restaurar el respaldo completo en el servidor de contingencia
RESTORE DATABASE GradosTítulos
FROM DISK = 'C:\Backups\GradosTítulos_Full.bak'
WITH
MOVE 'GradosTítulos_Data'
TO 'E:\Contingencia\GradosTítulos_Data.mdf',

MOVE 'GradosTítulos_Log'
TO 'E:\Contingencia\GradosTítulos_Log.ldf',

NORECOVERY,
STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTítulos
FROM DISK = 'C:\Backups\GradosTítulos_Diferencial.bak'
WITH NORECOVERY,
STATS = 10;
GO

-- Restaurar el respaldo del log de transacciones
RESTORE LOG GradosTítulos
FROM DISK = 'C:\Backups\GradosTítulos_Log.trn'
WITH RECOVERY,
STATS = 10;
GO

-- Verificar que la base quedó operativa
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTítulos';
GO

-- Consultar los archivos restaurados
USE GradosTítulos;
GO

SELECT
    name AS NombreLogico,
    physical_name AS RutaFisica,
    type_desc AS TipoArchivo
FROM sys.database_files;
GO
```

Justificación técnica de la solución aplicada

Se restauró la base de datos en una **instancia secundaria** utilizando los respaldos completo, diferencial y del log de transacciones. Esta estrategia permite disponer de un servidor de contingencia para garantizar la continuidad de los procesos académicos ante una falla del servidor principal.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las opciones **MOVE** para definir nuevas rutas físicas en el servidor de contingencia y la secuencia **FULL → DIFFERENTIAL → LOG** para asegurar una restauración correcta. Finalmente, se verificó el estado de la base y la ubicación de los archivos restaurados para confirmar que el servidor alternativo quedó listo para operar.

TEMA 3. Mantenimiento y Verificación de Copias de Seguridad

Proyecto 1: Verificación de Integridad de Backups

Enunciado

Verifique que todos los respaldos generados para **GradosTitulos** sean recuperables.

Script de la solución en T-SQL

```
USE master;
GO

-- Verificar el respaldo completo
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak';
GO

-- Verificar el respaldo diferencial
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak';
GO

-- Verificar el respaldo del Log de transacciones
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn';
GO

-- Consultar el historial de respaldos registrados
SELECT
    database_name AS BaseDatos,
    backup_start_date AS FechaInicio,
    backup_finish_date AS FechaFin,
    type AS TipoBackup,
    physical_device_name AS ArchivoRespaldo
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTitulos'
ORDER BY backup_finish_date DESC;
GO
```

Justificación técnica de la solución aplicada

Se utilizó **RESTORE VERIFYONLY** para comprobar que los archivos de respaldo pueden restaurarse correctamente sin necesidad de efectuar una restauración completa. Además, se consultó el historial de respaldos almacenado en la base de datos **msdb** para verificar las copias existentes.

Explicación de las buenas prácticas utilizadas en el proyecto

Se verificó cada tipo de respaldo (completo, diferencial y del log de transacciones) antes de utilizarlo en un proceso de recuperación. Asimismo, se consultó el historial de respaldos para mantener un control de las copias de seguridad disponibles y asegurar su disponibilidad.

Proyecto 2: Inventario Histórico de Copias de Seguridad

Enunciado

Construya un reporte que muestre todos los respaldos realizados durante el último mes.

Script de la solución en T-SQL

```

USE msdb;
GO

-- Reporte de respaldos realizados durante el último mes
SELECT
    bs.database_name AS BaseDatos,
    CASE bs.type
        WHEN 'D' THEN 'Respaldo Completo'
        WHEN 'I' THEN 'Respaldo Diferencial'
        WHEN 'L' THEN 'Respaldo del Log'
        ELSE 'Otro'
    END AS TipoRespaldo,
    bs.backup_start_date AS FechaInicio,
    bs.backup_finish_date AS FechaFin,
    bmf.physical_device_name AS ArchivoRespaldo,
    bs.backup_size / 1024 / 1024 AS TamañoMB
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTitulos'
AND bs.backup_finish_date >= DATEADD(MONTH, -1, GETDATE())
ORDER BY bs.backup_finish_date DESC;
GO

```

Justificación técnica de la solución aplicada

Se consultó el historial de respaldos almacenado en la base de datos **msdb**, filtrando únicamente las copias realizadas durante el último mes para la base de datos **GradosTitulos**, lo que permite conocer el historial reciente de respaldos.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó la fecha de finalización del respaldo para filtrar el período solicitado y se clasificó el tipo de respaldo (completo, diferencial o log), facilitando la administración y el seguimiento de las copias de seguridad.

Proyecto 3: Detección de Respaldos Obsoletos

Enunciado

Identifique copias de seguridad que excedan la política institucional de retención.

Script de la solución en T-SQL

```

USE msdb;
GO

-- Detectar respaldos con más de 30 días de antigüedad
SELECT
    bs.database_name AS BaseDatos,
    CASE bs.type
        WHEN 'D' THEN 'Respaldo Completo'
        WHEN 'I' THEN 'Respaldo Diferencial'
        WHEN 'L' THEN 'Respaldo del Log'
        ELSE 'Otro'
    END AS TipoRespaldo,
    bs.backup_finish_date AS FechaRespaldo,
    bmf.physical_device_name AS ArchivoRespaldo,
    DATEDIFF(DAY, bs.backup_finish_date, GETDATE()) AS DiasAntigüedad
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTitulos'
AND bs.backup_finish_date < DATEADD(DAY, -30, GETDATE())
ORDER BY bs.backup_finish_date;
GO

```

Justificación técnica de la solución aplicada

Se consultó el historial de respaldos almacenado en **msdb** para identificar las copias de seguridad cuya antigüedad supera los **30 días**, permitiendo detectar respaldos que ya no cumplen con la política institucional de retención.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó la función **DATEDIFF** para calcular la antigüedad de cada respaldo y un filtro por fecha para identificar únicamente las copias obsoletas, facilitando su revisión y posible eliminación conforme a las políticas de almacenamiento.

Proyecto 4: Sistema Automatizado de Verificación

Enunciado

Diseñe una solución que valide diariamente la integridad de todos los respaldos de **GradosTitulos**.

Script de la solución en T-SQL

```
USE GradosTitulos;
GO

-- Crear un procedimiento para verificar los respaldos
CREATE PROCEDURE SP_VerificarBackups
AS
BEGIN

    -- Verificar el respaldo completo
    RESTORE VERIFYONLY
    FROM DISK = 'C:\Backups\GradosTitulos_Full.bak';

    -- Verificar el respaldo diferencial
    RESTORE VERIFYONLY
    FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak';

    -- Verificar el respaldo del Log
    RESTORE VERIFYONLY
    FROM DISK = 'C:\Backups\GradosTitulos_Log.trn';

END;
GO

-- Ejecutar el procedimiento
EXEC SP_VerificarBackups;
GO

-- Consultar el historial de respaldos
SELECT
    database_name AS BaseDatos,
    backup_finish_date AS FechaRespaldo,
    type AS TipoBackup
FROM msdb.dbo.backupset
WHERE database_name = 'GradosTitulos'
ORDER BY backup_finish_date DESC;
GO
```

Justificación técnica de la solución aplicada

Se implementó un procedimiento almacenado que verifica la integridad de los respaldos mediante **RESTORE VERIFYONLY**, permitiendo automatizar la validación diaria de las copias de seguridad sin realizar una restauración completa.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centralizó la verificación en un único procedimiento almacenado para facilitar su programación mediante **SQL Server Agent**. Además, se consultó el historial de respaldos registrado en **msdb** para comprobar las copias disponibles y mantener un control permanente de su estado.

Proyecto 5: Sistema Automatizado de Validación de Backups

Enunciado

Diseñe un mecanismo que valide diariamente todos los respaldos de la base **GradosTitulos**.

Script de la solución en T-SQL

```
USE msdb;
GO

-- Crear el Job para validar los respaldos
EXEC sp_add_job
    @job_name = 'Validacion_Diaria_Backups_GradosTitulos';
GO

-- Agregar el paso que verifica los respaldos
EXEC sp_add_jobstep
    @job_name = 'Validacion_Diaria_Backups_GradosTitulos',
    @step_name = 'Verificar Integridad',
    @subsystem = 'TSQL',
    @database_name = 'master',
    @command = '
        RESTORE VERIFYONLY
        FROM DISK = ''C:\Backups\GradosTitulos_Full.bak'';

        RESTORE VERIFYONLY
        FROM DISK = ''C:\Backups\GradosTitulos_Diferencial.bak'';

        RESTORE VERIFYONLY
        FROM DISK = ''C:\Backups\GradosTitulos_Log.trn'';
    ';
GO

-- Crear la programación diaria
EXEC sp_add_schedule
    @schedule_name = 'Programacion_Diaria_Backups',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 010000;
GO

-- Asociar la programación al Job
EXEC sp_attach_schedule
    @job_name = 'Validacion_Diaria_Backups_GradosTitulos',
    @schedule_name = 'Programacion_Diaria_Backups';
GO

-- Habilitar el Job
EXEC sp_add_jobserver
    @job_name = 'Validacion_Diaria_Backups_GradosTitulos';
GO

-- Verificar los Jobs creados
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name = 'Validacion_Diaria_Backups_GradosTitulos';
GO
```

Justificación técnica de la solución aplicada

Se implementó un **Job de SQL Server Agent** que ejecuta diariamente la validación de los respaldos mediante **RESTORE VERIFYONLY**, garantizando que las copias de seguridad sean recuperables sin necesidad de restaurarlas.

Explicación de las buenas prácticas utilizadas en el proyecto

Se automatizó la validación mediante **SQL Server Agent**, evitando verificaciones manuales y asegurando un control periódico sobre la integridad de los respaldos. Además, se comprobó la creación del Job para confirmar que la automatización quedó correctamente configurada.

Proyecto 6: Auditoría Integral del Historial de Copias de Seguridad

Enunciado

La Dirección de TI solicita un reporte detallado de los respaldos ejecutados sobre **GradosTitulos** durante los últimos **90 días**.

Script de la solución en T-SQL

```
USE msdb;
GO

-- Reporte de respaldos de los últimos 90 días
SELECT
    bs.database_name AS BaseDatos,
    CASE bs.type
        WHEN 'D' THEN 'Respaldo Completo'
        WHEN 'I' THEN 'Respaldo Diferencial'
        WHEN 'L' THEN 'Respaldo del Log'
        ELSE 'Otro'
    END AS TipoRespaldo,
    bs.backup_start_date AS FechaInicio,
    bs.backup_finish_date AS FechaFin,
    bs.backup_size / 1024 / 1024 AS TamañoMB,
    bs.user_name AS Usuario,
    bmf.physical_device_name AS ArchivoRespaldo
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTitulos'
AND bs.backup_finish_date >= DATEADD(DAY, -90, GETDATE())
ORDER BY bs.backup_finish_date DESC;
GO

-- Resumen por tipo de respaldo
SELECT
    CASE type
        WHEN 'D' THEN 'Respaldo Completo'
        WHEN 'I' THEN 'Respaldo Diferencial'
        WHEN 'L' THEN 'Respaldo del Log'
        ELSE 'Otro'
    END AS TipoRespaldo,
    COUNT(*) AS CantidadRespaldos
FROM msdb.dbo.backupset
WHERE database_name = 'GradosTitulos'
AND backup_finish_date >= DATEADD(DAY, -90, GETDATE())
GROUP BY type;
GO
```

Justificación técnica de la solución aplicada

Se consultó el historial de respaldos almacenado en la base de datos **msdb** para generar un reporte de los últimos **90 días**, incluyendo información sobre el tipo de respaldo, fechas de ejecución, tamaño, usuario responsable y ubicación del archivo.

Explicación de las buenas prácticas utilizadas en el proyecto

Se filtró la información por el período solicitado y se agregó un resumen por tipo de respaldo para facilitar el control de las políticas de respaldo. Esto permite a la Dirección de TI supervisar el cumplimiento de la estrategia de copias de seguridad de la base **GradosTitulos**.

TEMA 4. Uso de BACKUP DATABASE y RESTORE DATABASE

Proyecto 1: Respaldo Manual de la Base GradosTitulos

Enunciado

Realice un respaldo completo utilizando únicamente instrucciones T-SQL.

Script de la solución en T-SQL

```
USE master;
GO

-- Realizar un respaldo completo de la base de datos
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
FORMAT,
INIT,
NAME = 'Respaldo Completo de GradosTitulos',
DESCRIPTION = 'Backup completo realizado mediante T-SQL',
COMPRESSION,
CHECKSUM,
STATS = 10;
GO

-- Verificar que el respaldo fue creado correctamente
RESTORE VERIFYONLY
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak';
GO

-- Consultar el historial del respaldo
SELECT
    database_name AS BaseDatos,
    backup_start_date AS FechaInicio,
    backup_finish_date AS FechaFin,
    backup_size / 1024 / 1024 AS TamañoMB,
    user_name AS Usuario
FROM msdb.dbo.backupset
WHERE database_name = 'GradosTitulos'
ORDER BY backup_finish_date DESC;
GO
```

Justificación técnica de la solución aplicada

Se realizó un respaldo completo de la base de datos **GradosTitulos** utilizando la instrucción **BACKUP DATABASE**, incorporando opciones de compresión, verificación y seguimiento del progreso para garantizar una copia de seguridad confiable.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron las opciones **COMPRESSION** para reducir el tamaño del respaldo, **CHECKSUM** para validar su integridad y **RESTORE VERIFYONLY** para comprobar que el archivo puede restaurarse correctamente. Finalmente, se consultó el historial de respaldos en **msdb** para confirmar la ejecución del proceso.

Proyecto 2: Restauración Completa desde Archivo BAK

Enunciado

Recupere la base de datos utilizando un archivo de respaldo existente.

Script de la solución en T-SQL

```

USE master;
GO

-- Restaurar la base de datos desde un archivo BAK
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
REPLACE,
RECOVERY,
STATS = 10;
GO

-- Verificar que la base de datos quedó disponible
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

```

Justificación técnica de la solución aplicada

Se restauró la base de datos **GradosTitulos** utilizando un archivo de respaldo **.BAK**, permitiendo recuperar completamente la información almacenada y dejar la base operativa para su uso.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó **WITH REPLACE** para sobrescribir la base existente cuando sea necesario y **WITH RECOVERY** para dejar la base de datos disponible al finalizar la restauración. Finalmente, se ejecutó **DBCC CHECKDB** para verificar la integridad física y lógica de la base restaurada.

Proyecto 3: Automatización de Backups y Restores

Enunciado

Diseñe procedimientos almacenados que automaticen respaldos y restauraciones.

Script de la solución en T-SQL

```

USE master;
GO

-- Procedimiento para realizar un respaldo completo
CREATE PROCEDURE SP_BackupGradosTitulos
AS
BEGIN
    BACKUP DATABASE GradosTitulos
    TO DISK = 'C:\Backups\GradosTitulos_Full.bak'
    WITH
        INIT,
        COMPRESSION,
        CHECKSUM,
        STATS = 10;
END;
GO

-- Procedimiento para restaurar la base de datos
CREATE PROCEDURE SP_RestoreGradosTitulos
AS
BEGIN
    RESTORE DATABASE GradosTitulos
    FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
    WITH
        REPLACE,
        RECOVERY,
        STATS = 10;
END;
GO

-- Ejecutar el respaldo
EXEC SP_BackupGradosTitulos;
GO

-- Ejecutar la restauración
EXEC SP_RestoreGradosTitulos;
GO

-- Verificar el estado de la base de datos
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

```

Justificación técnica de la solución aplicada

Se desarrollaron dos procedimientos almacenados para automatizar las operaciones de respaldo y restauración de la base de datos **GradosTitulos**, facilitando la ejecución repetitiva de estas tareas administrativas.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separaron las tareas de respaldo y restauración en procedimientos independientes para mejorar la organización y reutilización del código. Además, se utilizaron opciones como **COMPRESSION**, **CHECKSUM** y **STATS** para optimizar y supervisar el proceso, verificando finalmente que la base de datos quedara disponible después de la restauración.

Proyecto 4: Framework Institucional de Recuperación Enunciado

Construya un conjunto de scripts reutilizables para respaldo y restauración de todas las bases institucionales.

Script de la solución en T-SQL

```
USE master;
GO

-- Procedimiento reutilizable para realizar respaldos
CREATE PROCEDURE SP_BackupBaseDatos
    @BaseDatos SYSNAME,
    @RutaBackup NVARCHAR(300)
AS
BEGIN
    DECLARE @SQL NVARCHAR(MAX);

    SET @SQL =
        'BACKUP DATABASE [' + @BaseDatos + ']'
        'TO DISK = ''' + @RutaBackup + ''''
        'WITH INIT,'
        'COMPRESSION,'
        'CHECKSUM,'
        'STATS = 10;';

    EXEC (@SQL);
END;
GO

-- Procedimiento reutilizable para restaurar bases de datos
CREATE PROCEDURE SP_RestoreBaseDatos
    @BaseDatos SYSNAME,
    @RutaBackup NVARCHAR(300)
AS
BEGIN
    DECLARE @SQL NVARCHAR(MAX);

    SET @SQL =
        'RESTORE DATABASE [' + @BaseDatos + ']'
        'FROM DISK = ''' + @RutaBackup + ''''
        'WITH REPLACE,'
        'RECOVERY,'
        'STATS = 10;';

    EXEC (@SQL);
END;
GO

-- Ejemplo de respaldo de GradosTitulos
EXEC SP_BackupBaseDatos
    @BaseDatos = 'GradosTitulos',
    @RutaBackup = 'C:\Backups\GradosTitulos_Full.bak';
GO

-- Ejemplo de restauración de GradosTitulos
EXEC SP_RestoreBaseDatos
    @BaseDatos = 'GradosTitulos',
    @RutaBackup = 'C:\Backups\GradosTitulos_Full.bak';
GO
```

Justificación técnica de la solución aplicada

Se desarrolló un framework compuesto por procedimientos almacenados reutilizables que permiten realizar respaldos y restauraciones para cualquier base de datos institucional, evitando duplicar código y facilitando la administración del servidor.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizaron parámetros para indicar el nombre de la base de datos y la ruta del archivo de respaldo, haciendo que los procedimientos sean reutilizables. Además, se incorporaron las opciones **COMPRESSION**, **CHECKSUM** y **STATS** para optimizar y supervisar cada proceso.

Proyecto 5: Framework Automatizado de Respaldo Institucional

Enunciado

Desarrolle un procedimiento almacenado reutilizable que genere respaldos completos de **GradosTitulos** utilizando fecha y hora en el nombre del archivo.

Script de la solución en T-SQL

```
USE master;
GO

-- Crear procedimiento para generar respaldos con fecha y hora
CREATE PROCEDURE SP_BackupGradosTitulos_FechaHora
AS
BEGIN
    DECLARE @FechaHora NVARCHAR(30);
    DECLARE @RutaBackup NVARCHAR(300);
    DECLARE @SQL NVARCHAR(MAX);

    SET @FechaHora =
        CONVERT(VARCHAR(8), GETDATE(), 112) + '_' +
        REPLACE(CONVERT(VARCHAR(8), GETDATE(), 108), ':', '');

    SET @RutaBackup =
        'C:\Backups\GradosTitulos_' + @FechaHora + '.bak';

    SET @SQL =
        'BACKUP DATABASE GradosTitulos
        TO DISK = ''' + @RutaBackup + '''
        WITH
            COMPRESSION,
            CHECKSUM,
            INIT,
            STATS = 10;';

    EXEC(@SQL);

    PRINT 'Respaldo generado en: ' + @RutaBackup;
END;
GO

-- Ejecutar el procedimiento
EXEC SP_BackupGradosTitulos_FechaHora;
GO
```

Justificación técnica de la solución aplicada

Se desarrolló un procedimiento almacenado que genera automáticamente un respaldo completo de la base de datos **GradosTitulos**, incorporando la fecha y hora en el nombre del archivo para crear una copia única en cada ejecución.

Explicación de las buenas prácticas utilizadas en el proyecto

Se generó dinámicamente el nombre del archivo utilizando la fecha y hora del sistema, evitando sobrescribir respaldos anteriores. Además, se utilizaron las opciones **COMPRESSION**, **CHECKSUM** y **STATS** para optimizar el respaldo, verificar su integridad y mostrar el progreso de la operación.

Proyecto 6: Plataforma de Restauración Automatizada

Enunciado

Construya un procedimiento que permita restaurar automáticamente **GradosTitulos** desde cualquier archivo **BAK** seleccionado.

Script de la solución en T-SQL

```
USE master;
GO

-- Crear procedimiento para restaurar desde cualquier archivo BAK
CREATE PROCEDURE SP_RestaurarGradosTitulos
    @RutaBackup NVARCHAR(300)
AS
BEGIN
    DECLARE @SQL NVARCHAR(MAX);

    SET @SQL =
        'RESTORE DATABASE GradosTitulos
        FROM DISK = ''' + @RutaBackup + '''
        WITH
            REPLACE,
            RECOVERY,
            STATS = 10;';

    EXEC(@SQL);
END;
GO

-- Ejemplo de restauración
EXEC SP_RestaurarGradosTitulos
    @RutaBackup = 'C:\Backups\GradosTitulos_Full.bak';
GO

-- Verificar el estado de la base restaurada
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO
```

Justificación técnica de la solución aplicada

Se implementó un procedimiento almacenado que recibe como parámetro la ruta de cualquier archivo **.BAK**, permitiendo restaurar automáticamente la base de datos **GradosTitulos** desde el respaldo seleccionado por el administrador.

Explicación de las buenas prácticas utilizadas en el proyecto

Se parametrizó la ruta del archivo de respaldo para que el procedimiento pueda reutilizarse con diferentes copias de seguridad. Además, se utilizó **WITH REPLACE** y **WITH RECOVERY** para completar la restauración y se verificó la integridad de la base mediante **DBCC CHECKDB** al finalizar el proceso.

TEMA 5. Planes de Mantenimiento y Políticas de Retención

Proyecto 1: Creación de un Plan de Mantenimiento Semanal Enunciado

Creación de un Plan de Mantenimiento Semanal.

Script de la solución en T-SQL

```

USE msdb;
GO

-- Crear el Job de mantenimiento semanal
EXEC sp_add_job
    @job_name = 'Plan_Mantenimiento_Semanal_GradosTitulos';
GO

-- Agregar el paso del mantenimiento
EXEC sp_add_jobstep
    @job_name = 'Plan_Mantenimiento_Semanal_GradosTitulos',
    @step_name = 'Respaldo y Verificación',
    @subsystem = 'TSQL',
    @database_name = 'master',
    @command = '
        BACKUP DATABASE GradosTitulos
        TO DISK = ''C:\Backups\GradosTitulos_Semanal.bak''
        WITH INIT,
            COMPRESSION,
            CHECKSUM,
            STATS = 10;

        DBCC CHECKDB (''GradosTitulos'');
    ';
GO

-- Programar la ejecución semanal
EXEC sp_add_schedule
    @schedule_name = 'Mantenimiento_Semanal',
    @freq_type = 8,
    @freq_interval = 1,
    @active_start_time = 020000;
GO

-- Asociar el Job a la programación
EXEC sp_attach_schedule
    @job_name = 'Plan_Mantenimiento_Semanal_GradosTitulos',
    @schedule_name = 'Mantenimiento_Semanal';
GO

-- Habilitar el Job
EXEC sp_add_jobserver
    @job_name = 'Plan_Mantenimiento_Semanal_GradosTitulos';
GO

-- Verificar que el Job fue creado
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name = 'Plan_Mantenimiento_Semanal_GradosTitulos';
GO

```

Justificación técnica de la solución aplicada

Se implementó un **Plan de Mantenimiento Semanal** mediante un **Job de SQL Server Agent**, encargado de realizar un respaldo completo y verificar la integridad de la base de datos **GradosTitulos** de forma automática.

Explicación de las buenas prácticas utilizadas en el proyecto

Se programó la ejecución semanal del mantenimiento para reducir la intervención manual y asegurar la continuidad de las tareas administrativas. Además, se utilizaron las opciones **COMPRESSION** y **CHECKSUM** durante el respaldo y **DBCC CHECKDB** para validar la integridad de la base de datos.

Proyecto 2: Política de Retención de 90 Días

Enunciado

Implemente una política institucional para conservar respaldos durante **90 días**.

Script de la solución en T-SQL

```

USE msdb;
GO

-- Crear el Job para la política de retención
EXEC sp_add_job
    @job_name = 'Retencion_Backups_90_Dias';
GO

-- Agregar el paso para eliminar respaldos antiguos
EXEC sp_add_jobstep
    @job_name = 'Retencion_Backups_90_Dias',
    @step_name = 'Eliminar Backups Antiguos',
    @subsystem = 'CmdExec',
    @command = 'forfiles /P "D:\Backups" /M *.bak /D -90 /C "cmd /c del @path"';
GO

-- Programar la ejecución diaria
EXEC sp_add_schedule
    @schedule_name = 'Retencion_Diaria_90_Dias',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 010000;
GO

-- Asociar la programación al Job
EXEC sp_attach_schedule
    @job_name = 'Retencion_Backups_90_Dias',
    @schedule_name = 'Retencion_Diaria_90_Dias';
GO

-- Habilitar el Job
EXEC sp_add_jobserver
    @job_name = 'Retencion_Backups_90_Dias';
GO

-- Verificar el Job creado
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name = 'Retencion_Backups_90_Dias';
GO

```

Justificación técnica de la solución aplicada

Se implementó una política institucional de retención mediante un **Job de SQL Server Agent**, encargado de eliminar automáticamente los archivos de respaldo con una antigüedad superior a **90 días**, manteniendo el almacenamiento organizado y cumpliendo la política definida.

Explicación de las buenas prácticas utilizadas en el proyecto

Se automatizó la eliminación de respaldos antiguos mediante una programación diaria, evitando la acumulación innecesaria de archivos y reduciendo el consumo de espacio en disco. Además, se verificó la creación del Job para asegurar el correcto funcionamiento de la política de retención.

Proyecto 3: Plan de Mantenimiento Integral

Enunciado

Diseña un plan que incluya:

- Backups.
- Reorganización de índices.
- Actualización de estadísticas.
- Verificación de integridad.

Script de la solución en T-SQL


```

USE msdb;
GO

-- Crear el Job del Plan de Mantenimiento Integral
EXEC sp_add_job
    @job_name = 'Plan_Mantenimiento_Integral_GradosTitulos';
GO

-- Agregar el paso de mantenimiento
EXEC sp_add_jobstep
    @job_name = 'Plan_Mantenimiento_Integral_GradosTitulos',
    @step_name = 'Mantenimiento General',
    @subsystem = 'TSQL',
    @database_name = 'GradosTitulos',
    @command = '

-- Respaldo completo
BACKUP DATABASE GradosTitulos
TO DISK = ''D:\Backups\GradosTitulos_Mantenimiento.bak''
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;

-- Reorganizar índices
EXEC sp_MSforeachtable
    ''ALTER INDEX ALL ON ? REORGANIZE'';

-- Actualizar estadísticas
EXEC sp_updatestats;

-- Verificar integridad
DBCC CHECKDB (''GradosTitulos'');

';
GO

-- Programación semanal
EXEC sp_add_schedule
    @schedule_name = 'Mantenimiento_Integral_Semanal',
    @freq_type = 8,
    @freq_interval = 1,
    @active_start_time = 020000;
GO

-- Asociar el Job a la programación
EXEC sp_attach_schedule
    @job_name = 'Plan_Mantenimiento_Integral_GradosTitulos',
    @schedule_name = 'Mantenimiento_Integral_Semanal';
GO

-- Habilitar el Job
EXEC sp_add_jobserver
    @job_name = 'Plan_Mantenimiento_Integral_GradosTitulos';
GO

-- Verificar el Job creado
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name = 'Plan_Mantenimiento_Integral_GradosTitulos';
GO

```

Justificación técnica de la solución aplicada

Se diseñó un plan de mantenimiento integral que automatiza el respaldo de la base de datos, la reorganización de índices, la actualización de estadísticas y la verificación de integridad, mejorando el rendimiento y la disponibilidad de **GradosTitulos**.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centralizaron todas las tareas de mantenimiento en un único **Job de SQL Server Agent**, facilitando su administración y ejecución periódica. Además, se utilizaron herramientas nativas de SQL Server para optimizar el rendimiento y comprobar la integridad de la base de datos.

Proyecto 4: Gestión Corporativa de Retención

Enunciado

Diseña una política institucional de respaldo y retención para cinco años de operación.

Script de la solución en T-SQL

```

USE msdb;
GO

-- Job para generar el respaldo institucional
EXEC sp_add_job
    @job_name = 'Backup_Corporativo_GradosTitulos';
GO

EXEC sp_add_jobstep
    @job_name = 'Backup_Corporativo_GradosTitulos',
    @step_name = 'Respaldo Completo',
    @subsystem = 'TSQL',
    @database_name = 'master',
    @command = '
        BACKUP DATABASE GradosTitulos
        TO DISK = 'D:\Backups\GradosTitulos_Corporativo.bak'
        WITH INIT,
        COMPRESSION,
        CHECKSUM,
        STATS = 10;
    ';
GO

EXEC sp_add_schedule
    @schedule_name = 'Backup_Semanal_Corporativo',
    @freq_type = 8,
    @freq_interval = 1,
    @active_start_time = 020000;
GO

EXEC sp_attach_schedule
    @job_name = 'Backup_Corporativo_GradosTitulos',
    @schedule_name = 'Backup_Semanal_Corporativo';
GO

EXEC sp_add_jobserver
    @job_name = 'Backup_Corporativo_GradosTitulos';
GO

-- Job para eliminar respaldos con más de 5 años
EXEC sp_add_job
    @job_name = 'Retencion_5_Anios';
GO

EXEC sp_add_jobstep
    @job_name = 'Retencion_5_Anios',
    @step_name = 'Eliminar Backups Antiguos',
    @subsystem = 'CmdExec',
    @command = 'forfiles /P "D:\Backups" /M *.bak /D -1825 /C "cmd /c del @path";
GO

```

```

EXEC sp_add_schedule
    @schedule_name = 'Retencion_Mensual_5_Anios',
    @freq_type = 16,
    @freq_interval = 1,
    @active_start_time = 030000;
GO

EXEC sp_attach_schedule
    @job_name = 'Retencion_5_Anios',
    @schedule_name = 'Retencion_Mensual_5_Anios';
GO

EXEC sp_add_jobserver
    @job_name = 'Retencion_5_Anios';
GO

-- Verificar los Jobs creados
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name IN ('Backup_Corporativo_GradosTitulos', 'Retencion_5_Anios');
GO

```

Justificación técnica de la solución aplicada

Se implementó una política institucional que automatiza la generación de respaldos y la eliminación de copias con una antigüedad mayor a **cinco años**, garantizando la conservación de la información conforme a la política de retención establecida.

Explicación de las buenas prácticas utilizadas en el proyecto

Se separaron las tareas de respaldo y retención en **Jobs independientes**, facilitando su administración. Además, se programó la eliminación automática de respaldos obsoletos para optimizar el uso del almacenamiento sin afectar las copias que aún deben conservarse.

Proyecto 5: Implementación de un Plan Corporativo de Mantenimiento

Enunciado

Diseñe un plan de mantenimiento que incluya:

- Respaldo completo.
- Reorganización de índices.
- Actualización de estadísticas.
- Validación de integridad.

Script de la solución en T-SQL

```
USE msdb;
GO

-- Crear el Job de mantenimiento corporativo
EXEC sp_add_job
    @job_name = 'Plan_Corporativo_Mantenimiento_GradosTitulos';
GO

-- Agregar el paso de mantenimiento
EXEC sp_add_jobstep
    @job_name = 'Plan_Corporativo_Mantenimiento_GradosTitulos',
    @step_name = 'Mantenimiento Completo',
    @subsystem = 'TSQL',
    @database_name = 'GradosTitulos',
    @command = '

-- Respaldo completo
BACKUP DATABASE GradosTitulos
TO DISK = ''D:\Backups\GradosTitulos_Corporativo.bak''
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10;

-- Reorganización de índices
EXEC sp_MSforeachtable
    ''ALTER INDEX ALL ON ? REORGANIZE'';

-- Actualización de estadísticas
EXEC sp_updatestats;

-- Validación de integridad
DBCC CHECKDB (''GradosTitulos'');

';
GO

-- Programar la ejecución semanal
EXEC sp_add_schedule
    @schedule_name = 'Mantenimiento_Corporativo_Semanal',
    @freq_type = 8,
    @freq_interval = 1,
    @active_start_time = 010000;
GO

-- Asociar la programación al Job
EXEC sp_attach_schedule
    @job_name = 'Plan_Corporativo_Mantenimiento_GradosTitulos',
    @schedule_name = 'Mantenimiento_Corporativo_Semanal';
GO

-- Habilitar el Job
EXEC sp_add_jobserver
    @job_name = 'Plan_Corporativo_Mantenimiento_GradosTitulos';
GO

-- Verificar el Job creado
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name = 'Plan_Corporativo_Mantenimiento_GradosTitulos';
GO
```

Justificación técnica de la solución aplicada

Se implementó un plan de mantenimiento corporativo que automatiza el respaldo completo, la reorganización de índices, la actualización de estadísticas y la validación de integridad de la base de datos **GradosTitulos**, contribuyendo a mantener su rendimiento y disponibilidad.

Explicación de las buenas prácticas utilizadas en el proyecto

Se centralizaron todas las tareas de mantenimiento en un único Job de SQL Server Agent, facilitando su ejecución automática y reduciendo la intervención manual. Además, se utilizaron herramientas nativas de SQL Server para optimizar el rendimiento y garantizar la integridad de la base de datos.

Proyecto 6: Implementación de Política de Retención Automatizada

Enunciado

Diseñe una política institucional que elimine automáticamente respaldos con más de **180 días** de antigüedad.

Script de la solución en T-SQL

```
USE msdb;
GO

-- Crear el Job para la política de retención
EXEC sp_add_job
    @job_name = 'Retencion_Backups_180_Dias';
GO

-- Agregar el paso para eliminar respaldos con más de 180 días
EXEC sp_add_jobstep
    @job_name = 'Retencion_Backups_180_Dias',
    @step_name = 'Eliminar Backups Antiguos',
    @subsystem = 'CmdExec',
    @command = 'forfiles /P "D:\Backups" /M *.bak /D -180 /C "cmd /c del @path"';
GO

-- Programar la ejecución diaria
EXEC sp_add_schedule
    @schedule_name = 'Retencion_Diaria_180_Dias',
    @freq_type = 4,
    @freq_interval = 1,
    @active_start_time = 010000;
GO

-- Asociar la programación al Job
EXEC sp_attach_schedule
    @job_name = 'Retencion_Backups_180_Dias',
    @schedule_name = 'Retencion_Diaria_180_Dias';
GO

-- Habilitar el Job
EXEC sp_add_jobserver
    @job_name = 'Retencion_Backups_180_Dias';
GO

-- Verificar que el Job fue creado
SELECT
    name AS NombreJob,
    enabled AS Habilitado
FROM msdb.dbo.sysjobs
WHERE name = 'Retencion_Backups_180_Dias';
GO
```

Justificación técnica de la solución aplicada

Se implementó una política institucional de retención mediante un **Job de SQL Server Agent**, encargado de eliminar automáticamente los archivos de respaldo que superen los **180 días** de antigüedad, optimizando el uso del almacenamiento.

Explicación de las buenas prácticas utilizadas en el proyecto

Se automatizó la eliminación de respaldos antiguos mediante una ejecución diaria, evitando la acumulación innecesaria de archivos y asegurando el cumplimiento de la política de retención institucional. Además, se verificó la creación del Job para confirmar que la automatización quedó correctamente configurada.

TEMA 6. Restauración Punto en el Tiempo

Proyecto 1: Recuperación Antes de una Eliminación Accidental

Enunciado

Un operador eliminó registros de expedientes a las **10:15 a.m.** Restaure la base exactamente a las **10:14 a.m.**

Script de la solución en T-SQL

```

USE master;
GO

-- Colocar la base en modo de restauración
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
    NORECOVERY,
    REPLACE,
    STATS = 10;
GO

-- Restaurar el respaldo del log hasta las 10:14 a.m.
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH
    STOPAT = '2026-06-28T10:14:00',
    RECOVERY,
    STATS = 10;
GO

-- Verificar que la base quedó operativa
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

```

Justificación técnica de la solución aplicada

Se restauró la base de datos utilizando un respaldo completo y posteriormente el respaldo del log de transacciones con la opción **STOPAT**, permitiendo recuperar el estado de **GradosTitulos** exactamente a las **10:14 a.m.**, antes de la eliminación accidental.

Explicación de las buenas prácticas utilizadas en el proyecto

Se utilizó **NORECOVERY** para permitir la restauración del log de transacciones y **RECOVERY** para dejar la base disponible al finalizar el proceso. Finalmente, se ejecutó **DBCC CHECKDB** para comprobar la integridad de la base restaurada.

Proyecto 2: Recuperación de Cambios Incorrectos

Enunciado

Un proceso masivo actualizó incorrectamente estados de trámites.

Script de la solución en T-SQL

```

USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
    REPLACE,
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el log de transacciones hasta antes del cambio incorrecto
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH
    STOPAT = '2026-06-28T14:29:59',
    RECOVERY,
    STATS = 10;
GO

-- Verificar el estado de la base
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Comprobar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

```

Justificación técnica de la solución aplicada

Se restauró la base de datos utilizando el respaldo completo y el respaldo del log de transacciones hasta el instante previo a la actualización incorrecta, recuperando los estados originales de los trámites.

Explicación de las buenas prácticas utilizadas en el proyecto

Se empleó la opción **STOPAT** para realizar una recuperación punto en el tiempo, minimizando la pérdida de información. Además, se verificó el estado e integridad de la base de datos mediante **DBCC CHECKDB** después de finalizar la restauración.

Proyecto 3: Recuperación Selectiva de Incidente Crítico

Enunciado

Un error en producción afectó las tablas **Tramite** y **Evaluacion**. Diseñe una recuperación punto en el tiempo minimizando la indisponibilidad.

Script de la solución en T-SQL

```

USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
    NORECOVERY,
    REPLACE,
    STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el log hasta antes del incidente
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH
    STOPAT = '2026-06-28T15:29:59',
    RECOVERY,
    STATS = 10;
GO

-- Verificar que la base quedó disponible
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

```

Justificación técnica de la solución aplicada

Se utilizó una estrategia de recuperación compuesta por un respaldo completo, un respaldo diferencial y el log de transacciones con la opción **STOPAT**, permitiendo restaurar la base de datos al instante previo al incidente y reduciendo el tiempo de recuperación.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó la secuencia correcta de restauración (**Full** → **Diferencial** → **Log**) para minimizar la indisponibilidad del sistema. Finalmente, se verificó el estado e integridad de la base de datos mediante **DBCC CHECKDB** para asegurar una recuperación exitosa.

Proyecto 4: Simulación de Recuperación Forense Institucional Enunciado

Se detectó una modificación maliciosa realizada durante una ventana específica de tiempo. Realice una recuperación forense que permita restaurar la base **GradosTitulos** exactamente al momento anterior al incidente y documentar todo el procedimiento.

Script de la solución en T-SQL

```

USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
    REPLACE,
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el log de transacciones hasta antes del incidente
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH
    STOPAT = '2026-06-28T16:29:59',
    RECOVERY,
    STATS = 10;
GO

-- Verificar el estado de la base restaurada
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

-- Documentar la recuperación realizada
SELECT
    GETDATE() AS FechaRecuperacion,
    'Recuperación forense completada correctamente.' AS Resultado;
GO

```

Justificación técnica de la solución aplicada

Se realizó una recuperación forense restaurando el respaldo completo, el respaldo diferencial y el log de transacciones hasta el momento inmediatamente anterior al incidente mediante **STOPAT**, permitiendo recuperar la base de datos sin incluir las modificaciones maliciosas.

Explicación de las buenas prácticas utilizadas en el proyecto

Se siguió la secuencia correcta de restauración (**Full** → **Diferencial** → **Log**) para minimizar la pérdida de información. Además, se verificó la integridad de la base con **DBCC CHECKDB** y se registró la fecha de la recuperación como evidencia del procedimiento realizado.

Proyecto 5: Recuperación Exacta de Expedientes Eliminados

Enunciado

A las **11:45 a.m.** se eliminaron accidentalmente registros de expedientes de titulación. Recupere la base exactamente a las **11:44:59 a.m.**

Script de la solución en T-SQL


```

USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Full.bak'
WITH
    REPLACE,
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Diferencial.bak'
WITH
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el log de transacciones hasta las 11:44:59 a.m.
RESTORE LOG GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Log.trn'
WITH
    STOPAT = '2026-06-28T11:44:59',
    RECOVERY,
    STATS = 10;
GO

-- Verificar que la base quedó operativa
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

```

Justificación técnica de la solución aplicada

Se restauró la base de datos utilizando un respaldo completo, un respaldo diferencial y el log de transacciones hasta las **11:44:59 a.m.** mediante la opción **STOPAT**, recuperando los expedientes eliminados antes de que ocurriera el incidente.

Explicación de las buenas prácticas utilizadas en el proyecto

Se siguió la secuencia de restauración **Full → Diferencial → Log**, reduciendo el tiempo de recuperación y la pérdida de información. Además, se verificó el estado de la base de datos y su integridad mediante **DBCC CHECKDB** al finalizar el proceso.

Proyecto 6: Recuperación Forense de una Modificación Maliciosa

Enunciado

Se detectó una modificación no autorizada en las tablas del esquema **Documento** relacionadas con resoluciones y diplomas. Realice una recuperación punto en el tiempo que permita restaurar el estado exacto de la base antes de la alteración.

Script de la solución en T-SQL

```

USE master;
GO

-- Restaurar el respaldo completo
RESTORE DATABASE GradosTitulos
FROM DISK = 'D:\Backups\GradosTitulos_Full.bak'
WITH
    REPLACE,
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el respaldo diferencial
RESTORE DATABASE GradosTitulos
FROM DISK = 'D:\Backups\GradosTitulos_Diferencial.bak'
WITH
    NORECOVERY,
    STATS = 10;
GO

-- Restaurar el log de transacciones hasta antes de la modificación no autorizada
RESTORE LOG GradosTitulos
FROM DISK = 'D:\Backups\GradosTitulos_Log.trn'
WITH
    STOPAT = '2026-06-28T17:29:59',
    RECOVERY,
    STATS = 10;
GO

-- Verificar que la base quedó operativa
SELECT
    name AS BaseDatos,
    state_desc AS Estado
FROM sys.databases
WHERE name = 'GradosTitulos';
GO

-- Verificar la integridad de la base restaurada
DBCC CHECKDB ('GradosTitulos');
GO

-- Registrar la fecha de la recuperación
SELECT
    GETDATE() AS FechaRecuperacion,
    'Recuperación punto en el tiempo completada correctamente.' AS Resultado;
GO

```

Justificación técnica de la solución aplicada

Se realizó una recuperación punto en el tiempo restaurando el respaldo completo, el respaldo diferencial y el log de transacciones hasta el instante anterior a la modificación no autorizada, recuperando el estado correcto de las tablas del esquema **Documento**.

Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicó la secuencia **Full → Diferencial → Log** utilizando la opción **STOPAT** para minimizar la pérdida de información. Finalmente, se verificó la integridad de la base con **DBCC CHECKDB** y se registró la fecha de la recuperación como evidencia del procedimiento ejecutado.